Understanding Embeddings

From Token Lookup Tables to Modern Semantic Representations

A short technical handbook for AI engineers, developers, and serious learners

@techNmak

Handbook 08 - Understanding AI Series

ORIENTATION

What this handbook will make clear

"Embedding" is one of the most overloaded words in modern AI. It can mean a row in a learned token table, a contextual hidden state inside a Transformer, a pooled sentence vector, a document representation used for retrieval, or even a vector shared across text and images.

This handbook separates those meanings and connects them. By the end, you should be able to explain:

- why a token lookup table is a learned parameter matrix rather than a database of definitions;
- how static word vectors differ from contextual token states;
- how variable-length text becomes a fixed-size vector through pooling or model-specific readout;
- why cosine similarity, dot product, normalization, and Euclidean distance are not interchangeable choices;
- how contrastive learning shapes a space so useful neighbors become close and negatives become separable;
- why retrieval models often treat queries and documents differently;
- what embedding dimension changes, and when truncation is mathematically justified;
- why a vector space is only meaningful together with the model, preprocessing, objective, and similarity function that created it.

Prerequisites

Basic comfort with vectors, matrices, dot products, softmax, and Transformer hidden states is enough. The earlier handbooks on tokenization and attention are useful context, but this handbook rebuilds the embedding mechanism from first principles.

Scope

This is a handbook about learned representations. It touches dense retrieval, RAG, multimodal models, and vector indexes only to explain how embeddings are produced and compared. HNSW, hybrid search, reranking, chunking, and full retrieval-system design belong in their own handbooks.

Notation used throughout

V = vocabulary size; T = sequence length; d_model = Transformer hidden/token-embedding width; d = a generic representation width; d_e = the final text-embedding dimension when it differs from the model hidden width. Sections define any additional local symbols where they appear.

Research anchors: [1], [4], [7], [9], [10], [11], [14], [18]

1 - WHAT AN EMBEDDING IS

A learned map into a vector space

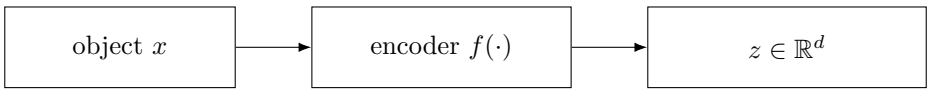
An embedding is a function that maps an object into a finite-dimensional vector:

$$f(x) \in \mathbb{R}^d.$$

The object x might be a token ID, a word, a sentence, a document, an image, a code snippet, or something else entirely. The useful part is not that the output is "a list of numbers." The useful part is that training makes relationships in the vector space support a downstream objective.

For a vocabulary token, the mapping may be as simple as selecting a learned row from a matrix. For a sentence, it may require a Transformer plus a pooling or readout rule. For retrieval, the training objective may explicitly encourage relevant query-document pairs to have high similarity.

An embedding therefore has no universal semantic meaning independent of its training process. Coordinate 417 is not automatically "negation" and coordinate 901 is not "topic." The representation is distributed: useful information is generally encoded across many dimensions and directions.



Mental model. An embedding is a learned representation, not a semantic coordinate system handed to the model in advance.

Research anchors: [1], [2], [3], [13]

2 - WHY DISTRIBUTED REPRESENTATIONS

Generalize by sharing statistical strength

A one-hot representation gives each vocabulary item its own independent axis. If the vocabulary has V entries, token i can be represented by a vector in $\mathbb{R}^V$ that is zero everywhere except position i.

That encoding uniquely identifies the token, but it does not make two related tokens close. “cat” and “dog” are as orthogonal as “cat” and “compiler.” A learned dense representation instead gives each item a vector in a much smaller space $\mathbb{R}^d$, with $d \ll V$ in many models.

Bengio et al. framed learned continuous word representations as a way to fight the curse of dimensionality in language modeling: training examples can influence predictions for nearby representations rather than only exact discrete sequences. Later word-vector methods such as word2vec and GloVe made large-scale learning of useful geometric structure much cheaper. [1–3]

The word *distributed* matters. A concept is not normally assigned one dedicated dimension. Each vector uses many coordinates, and each coordinate can participate in representing many items.

Important. Dense does not automatically mean semantic. A random dense table is still just a random table. The geometry becomes useful because an objective and data shape it during training.

Research anchors: [1], [2], [3]

3 - THE TOKEN EMBEDDING MATRIX

The first learned representation an LLM sees

Let a tokenizer produce token IDs in $\{0, \ldots, V-1\}$. A standard token-embedding layer stores a matrix

$$E \in \mathbb{R}^{V \times d_{\text{model}}}.$$

Row E_i is the learned vector associated with token ID i.

For a sequence of token IDs

$$(t_1, t_2, \ldots, t_n),$$

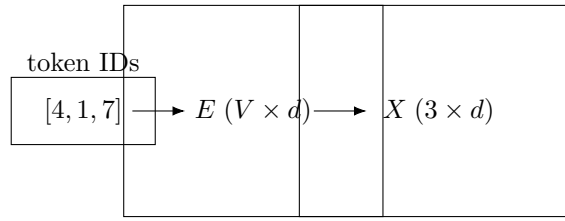
the embedding lookup returns

$$X = [E_{t_1}; E_{t_2}; \ldots; E_{t_n}] \in \mathbb{R}^{n \times d_{\text{model}}}.$$

In a batch, an implementation commonly returns a tensor with logical shape

$$B \times T \times d_{\text{model}}.$$

PyTorch describes `nn.Embedding` directly as a lookup table that stores embeddings for a fixed dictionary and embedding size. [19]



The embedding row is a model parameter. It is not the tokenizer's definition of the token, and token IDs themselves do not contain geometry. IDs merely select rows.

Mental model. Tokenization chooses discrete symbols. The embedding matrix turns those symbols into learned continuous vectors.

Research anchors: [4], [7], [19]

4 - LOOKUP IS A MATRIX OPERATION

One-hot multiplication and row selection are equivalent

Embedding lookup is often implemented as indexing because it is efficient, but it can be understood as ordinary matrix multiplication.

Suppose

$$E = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & 1 \\ 3 & 1 & 0 \\ 2 & 2 & 2 \end{bmatrix}$$

contains four token embeddings of dimension three. Token ID 2 corresponds to the one-hot row vector

$$e_2^\top = [0, 0, 1, 0].$$

Then

$$e_2^\top E = [3, 1, 0],$$

which is exactly row 2 of E under zero-based indexing.

For a sequence, stack one-hot rows into $O \in \mathbb{R}^{T \times V}$. Then

$$X = OE \in \mathbb{R}^{T \times d}.$$

A real implementation does not usually construct the enormous sparse one-hot matrix. It gathers only the requested rows.

This equivalence is useful because it removes some mystery from an "embedding layer." It is a learned linear map from a discrete basis into a lower-dimensional continuous space, evaluated efficiently by indexing.

Important. Lookup does not average, search, or retrieve the nearest token. Given an ID, the layer selects the corresponding parameter row exactly.

Research anchors: [1], [19]

5 - PARAMETER COUNT AND MEMORY

Vocabulary size times embedding width

The token embedding table contains

$$V\, d_{\text{model}}$$

parameters. This can be a nontrivial fraction of a smaller language model, especially when the vocabulary is large.

Take an illustrative configuration:

$$V = 32{,}768, \qquad d_{\text{model}} = 4096.$$

Then

$$Vd = 134{,}217{,}728$$

embedding parameters. At two bytes per parameter, the raw parameter payload is

$$268{,}435{,}456 \text{ bytes} = 256 \text{ MiB}.$$

If a model also used a separate untied output matrix of the same shape, that matrix would require another 256 MiB of two-byte parameter storage in this example.

This calculation is only the parameter payload. Training memory can additionally include gradients, optimizer states, higher-precision master weights, sharding metadata, and allocator overhead.

Vocabulary design therefore interacts with model size. Adding tokens is not free: every new vocabulary row adds d_{model} learned parameters to an untied input table.

Important. Embedding dimension and vocabulary size are different axes. Increasing V adds rows. Increasing d makes every row wider and also changes the hidden width if the token embedding feeds the model directly.

Research anchors: [4], [5], [19]

6 - HOW EMBEDDINGS ARE LEARNED

Rows move because downstream loss sends gradients back

An embedding table does not need a special "semantic" loss to become useful. If it participates in a differentiable model, the task loss can update the selected rows through backpropagation.

Suppose token ID t selects row E_t. The rest of the network computes a loss $\mathcal{L}$. Backpropagation provides

$$\frac{\partial \mathcal{L}}{\partial E_t}.$$

An optimizer then updates that row along with the model's other parameters. Rows for tokens not used in the current batch may receive no direct lookup gradient in a simple embedding layer, although weight tying or other uses of the matrix can create additional gradient paths.

Over many examples, tokens that participate in similar predictive contexts can be pushed toward representations that support similar downstream computations. But there is no requirement that Euclidean neighborhoods become human-interpretable unless the objective makes that geometry useful.

Early neural language models explicitly learned distributed word vectors jointly with the probability model. Modern Transformers do the same at much larger scale: token embeddings are learned end-to-end with the rest of the network. [1,4]

Mental model. The embedding table is not pretrained knowledge glued onto the model. In an end-to-end model, it is one set of parameters among many, shaped by the same training signal.

Research anchors: [1], [4], [5]

7 - STATIC WORD EMBEDDINGS

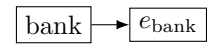
Word2vec and GloVe made vector geometry practical

Before contextual Transformers, a common representation assigned one vector to each word type. Word2vec introduced efficient predictive architectures for learning continuous word vectors from local contexts, including CBOW and skip-gram. GloVe instead trained a global log-bilinear regression model on word-word co-occurrence statistics. [2,3]

The details differ, but both families helped establish a durable idea: distributional regularities in text can be encoded as useful geometry in a learned vector space.

Static embeddings made operations such as nearest-neighbor search, similarity, clustering, and feature transfer inexpensive. They also exposed striking regularities in differences between vectors, although analogy behavior should not be mistaken for a universal algebra of meaning.

The key limitation is in the word *static*. A type such as "bank" receives the same base vector whether the sentence concerns money or a river. Context can only be incorporated later by another model.



river bank same type vector

Important. The famous linear regularities of some word-vector spaces are empirical properties of learned representations, not a guarantee that every semantic relation is a clean vector offset.

Research anchors: [2], [3]

8 - WHY STATIC WORD VECTORS ARE NOT ENOUGH

Meaning changes with context

Natural language is full of polysemy and context dependence. A single vector per word type must compress every use of that type into one representation.

Contextual representation models changed the unit of representation. ELMo generated word representations as functions of the entire input sentence using bidirectional language-model states. BERT then learned deep bidirectional Transformer representations by conditioning each token on both left and right context during pretraining. [6,7]

Now the two occurrences of "bank" in

- "She deposited cash at the bank."
- "They sat on the river bank."

can begin from the same token embedding row yet end with different contextual hidden states after the model processes their surroundings.

That distinction is foundational for modern embedding terminology:

Static token/type embedding	A learned vector associated with a discrete vocabulary item.
Contextual token representation	A hidden state produced for a token occurrence after contextual computation.

Mental model. The token table supplies a starting representation. The Transformer turns that starting point into a context-dependent state.

Research anchors: [6], [7]

9 - CONTEXTUAL EMBEDDINGS

The same token ID can produce different hidden vectors

Let token t_i have base embedding e_{t_i}. A Transformer layer stack computes contextual states

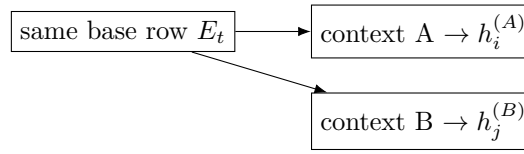
$$(h_1, \ldots, h_T) = F(e_{t_1}, \ldots, e_{t_T}; \theta).$$

Although the same token ID selects the same base row e_t wherever it appears, h_i depends on surrounding tokens, position treatment, attention, feed-forward computation, normalization, and the layer being inspected.

It is therefore imprecise to say "the embedding of token X" without specifying which representation is meant. Possible meanings include:

- the input lookup vector E_t;
- the hidden state for one occurrence at layer ℓ;
- a pooled text embedding derived from many token states;
- an output/unembedding row used to score vocabulary items.

Contextual representations are also layer-specific. The final hidden state is not interchangeable with an intermediate layer simply because both have the same dimensionality.



Important. "Contextual embedding" is common language, but the object is usually a hidden representation produced by a contextualizer, not a row stored in the input embedding table.

Research anchors: [6], [7]

10 - TOKEN EMBEDDINGS VS HIDDEN STATES

Do not collapse three different vector types

Modern language models expose several vectors with the same width, which makes them easy to confuse.

	Token embedding	**Contextual state**	**Text embedding**
Input	token ID	sequence states	text / token states
Context dependent?	no	yes	usually yes
One per	vocab item	token occurrence	sentence/document/etc.
Typical role	model input	internal computation	similarity/retrieval/task feature

The first is a parameter lookup. The second is an activation. The third is a representation deliberately exposed for comparing larger units such as sentences or documents.

Calling all three "embeddings" is not necessarily wrong, but it becomes dangerous when reasoning about memory, training, similarity, or reuse. A vocabulary embedding matrix lives in the model parameters. Contextual states are produced per input. A document embedding can often be precomputed and stored outside the model.

Mental model. Always ask: *what is being embedded, by which function, at which layer or readout, and for what objective?*

Research anchors: [4], [7], [9], [10]

11 - POSITION AND INPUT REPRESENTATIONS

Token identity is only one ingredient

A Transformer needs some way to represent sequence order in addition to token identity. The original Transformer added positional encodings to token embeddings before the first layer. BERT used learned token, segment, and position embeddings. Other modern models use RoPE inside the attention computation instead of adding a position vector to the input. [4,7]

So a general input representation should not be hard-coded as

$$\text{token embedding} + \text{position embedding}$$

for every architecture.

The durable distinction is:

- **token embedding**: represents the discrete vocabulary item;
- **position mechanism**: gives the network information about order or relative position;
- **contextualization**: mixes information across the sequence through the network.

In the original Transformer, token embeddings are multiplied by $\sqrt{d_{\text{model}}}$ before positional encodings are added. That scaling is an architecture-specific design choice, not part of the universal definition of embedding lookup. [4]

Important. Do not say RoPE "changes the token embedding." Standard RoPE acts on Q/K inside attention; it is a positional mechanism surrounding the contextual computation, not a replacement vocabulary table.

Research anchors: [4], [7]

12 - OUTPUT EMBEDDINGS AND WEIGHT TYING

The vocabulary matrix can appear on both sides of the model

A language model must eventually convert a hidden state into logits over vocabulary items. With row-vector convention, an untied output projection can be written

$$\ell = hW_{\text{out}}, \qquad W_{\text{out}} \in \mathbb{R}^{d \times V}.$$

Press and Wolf analyzed the output matrix itself as a word embedding and recommended tying input and output embeddings in language models. [5]

If the input table is $E \in \mathbb{R}^{V \times d}$ and weights are tied, the logits can be expressed as

$$\ell = hE^{\top}$$

(up to architecture-specific biases or scaling).

Weight tying reduces parameter count because the model no longer stores two independent matrices with the same vocabulary-by-width shape. It also couples their learning: the same parameter rows participate in both representing input symbols and scoring output symbols.

The Transformer paper shared the target input embedding and pre-softmax linear transformation weights, and additionally shared source/target embeddings in its translation setup. [4]

Important. "Embedding matrix" can refer to the input lookup table, the output vocabulary projection viewed as an embedding, or a tied matrix serving both roles. State which one you mean.

Research anchors: [4], [5]

13 - FROM TOKENS TO ONE TEXT VECTOR

Variable-length sequence in, fixed-size representation out

Retrieval and similarity systems usually need one vector per sentence, passage, or document. A Transformer, however, naturally produces one hidden state per token:

$$H = [h_1; \ldots; h_T] \in \mathbb{R}^{T \times d}.$$

A text-embedding model therefore needs a readout rule

$$z = g(H, \text{mask}) \in \mathbb{R}^{d_e}.$$

Common choices include mean pooling over valid tokens, a designated classification token, the final non-padding token, a learned pooling head, or a model-specific projection.

Sentence-BERT explicitly adds a pooling operation to BERT/RoBERTa outputs and studied mean, max, and CLS pooling. Its default in the original paper was mean pooling. [9]

Modern decoder-based embedding models can use a different readout. Qwen3 Embedding appends an EOS token and takes the final-layer hidden state at EOS as the text representation. [18]

There is no universal "correct pooling." The readout is part of the trained model definition.

Mental model. A sentence embedding is not simply "the BERT vector" or "the LLM vector." It is the output of an encoder plus a specific readout rule, usually trained or validated for the intended similarity task.

Research anchors: [9], [14], [18], [20]

14 - POOLING

Mask padding and preserve the model's intended readout

For mean pooling with hidden states h_i and a binary mask $m_i \in \{0, 1\}$,

$$z = \frac{\sum_{i=1}^{T} m_i h_i}{\sum_{i=1}^{T} m_i}.$$

The mask matters because padding tokens should not silently pull the vector toward a padding representation.

Consider three valid token states and one padded position:

$$h_1 = [1, 0],\ h_2 = [0, 2],\ h_3 = [2, 1],\ h_4 = [9, 9], \quad m = [1, 1, 1, 0].$$

Masked mean pooling gives

$$z = \frac{[1, 0] + [0, 2] + [2, 1]}{3} = [1, 1].$$

Including padding would instead produce $[3, 3]$, a completely different vector.

Pooling can also be model-specific. Sentence Transformers currently supports strategies including CLS, max, mean, weighted mean, and last-token pooling. [20]

Important. Pooling is not a harmless post-processing choice. If a model was trained with a specific readout, changing it can move you into a representation space the training objective never optimized.

Research anchors: [9], [14], [18], [20]

15 - RAW LM STATES ARE NOT AUTOMATICALLY GOOD SEMANTIC EMBEDDINGS

Generation objectives and similarity objectives are different

A pretrained language model can produce hidden states for any text, but that does not guarantee that a simple pooled vector has useful cosine geometry for semantic similarity.

Sentence-BERT showed this sharply for the BERT models and evaluation setup in its 2019 paper: directly averaging BERT outputs or using the CLS output performed poorly on semantic textual similarity compared with an encoder fine-tuned specifically for sentence embeddings. The point is not the old benchmark number. The point is the objective mismatch. [9]

A generative or masked-language-model objective trains hidden states to support token prediction and the model's internal computation. A retrieval embedding objective instead cares about pairwise geometry: relevant texts should score higher than irrelevant texts under a chosen similarity rule.

Modern LLM-based embedding models therefore fine-tune foundation models for embedding tasks rather than assuming the base model's hidden states already form an ideal retrieval space. Qwen3 Embedding, for example, trains the foundation-model backbone with a contrastive objective and a defined readout. [18]

Important. "Take the last hidden state and average it" is a baseline, not a universal recipe for high-quality embeddings.

Research anchors: [9], [11], [18], [21]

16 - SIAMESE AND BI-ENCODER MODELS

Encode items independently, compare them cheaply

A bi-encoder maps two inputs independently into vectors:

$$z_q = f_q(q), \qquad z_d = f_d(d),$$

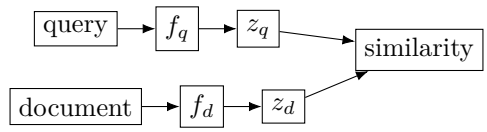
then computes a decomposable similarity such as

$$s(q, d) = z_q^\top z_d.$$

The encoders may share parameters or be separate. Sentence-BERT uses siamese/triplet-style structures to make independent sentence representations comparable. DPR uses distinct question and passage encoders, both based on BERT in the original work. [9,10]

The crucial systems property is decomposability. Document vectors can be precomputed without knowing the future query. At query time, only the query needs a fresh forward pass before vector search.

A cross-encoder is different: it processes the pair jointly, which can model richer token-level interactions but prevents independent precomputation of document scores for every possible query.



Mental model. Embedding models trade some pairwise interaction richness for reusable representations and scalable comparison.

Research anchors: [9], [10], [14]

17 - SIMILARITY: DOT PRODUCT, COSINE, AND L2

The metric is part of the model contract

Three common comparisons are

$$\text{dot}(u, v) = u^\top v,$$

$$\cos(u, v) = \frac{u^\top v}{\|u\|_2 \|v\|_2},$$

$$d_2(u, v) = \|u - v\|_2.$$

Dot product depends on both direction and vector magnitude. Cosine removes magnitude and compares angle. Euclidean distance measures geometric separation in the original scale.

If vectors are L2-normalized so that $\|u\| = \|v\| = 1$, then

$$u^\top v = \cos(u, v)$$

and

$$\|u - v\|_2^2 = 2 - 2u^\top v.$$

Therefore, for unit vectors, maximizing cosine/dot product and minimizing Euclidean distance induce the same ranking.

DPR deliberately used dot product. SimCSE's standard objective uses cosine similarity. E5 uses cosine scaled by temperature. These are training choices, not merely interchangeable query-time switches. [10,11,14]

Important. The safest production rule is to use the similarity function and normalization convention specified by the embedding model. Changing them can change rankings even if the vector dimensions are unchanged.

Research anchors: [10], [11], [14], [20]

18 - NORMALIZATION

Unit length changes what the score can encode

L2 normalization maps

$$z \mapsto \hat{z} = \frac{z}{\|z\|_2}.$$

After normalization, every nonzero vector lies on the unit hypersphere. Magnitude no longer contributes to dot-product similarity.

This can be desirable when the training objective is angular. It can also be wrong if the model was trained to use vector norm as part of the score.

Sentence Transformers explicitly notes that dot product on normalized embeddings is equivalent to cosine similarity, while avoiding redundant re-normalization at comparison time. [20]

Normalization also interacts with dimensional truncation. Matryoshka Representation Learning notes that when normalized representations are used, normalization should be handled independently for each nested dimension rather than truncating a normalized full vector and assuming the prefix remains unit length. [15]

A practical invariant is therefore:

encode convention + normalization convention + metric convention must agree.

Important. Do not normalize only queries or only documents unless the model explicitly defines that asymmetric scoring rule.

Research anchors: [11], [15], [20]

19 - WORKED GEOMETRY EXAMPLE

Same direction, different magnitude

Take

$$q = [3, 4], \qquad d_1 = [6, 8], \qquad d_2 = [4, 3].$$

Their norms are

$$\|q\| = 5, \quad \|d_1\| = 10, \quad \|d_2\| = 5.$$

Dot products are

$$q^\top d_1 = 50, \qquad q^\top d_2 = 24.$$

Cosine similarities are

$$\cos(q, d_1) = \frac{50}{5 \cdot 10} = 1,$$

$$\cos(q, d_2) = \frac{24}{5 \cdot 5} = 0.96.$$

The Euclidean distances are

$$\|q - d_1\| = 5, \qquad \|q - d_2\| = \sqrt{2} \approx 1.414.$$

So d_1 is perfectly aligned with q but farther away in raw Euclidean space because its magnitude is doubled. Dot product strongly favors it because magnitude also contributes.

After normalizing all vectors, cosine and dot product agree, and Euclidean distance becomes a monotonic transformation of the same angle.

Mental model. "Nearest" is undefined until you specify the representation scaling and the metric.

Research anchors: [10], [20]

20 - CONTRASTIVE LEARNING

Shape the space with positives and negatives

Modern text embedding models are often trained with a contrastive objective. The intuition is simple: increase similarity for related pairs and reduce it relative to competing negatives.

A common in-batch objective for anchor x_i, positive x_i^+, batch size N, similarity s, and temperature τ is

$$\ell_i = -\log \frac{\exp(s(z_i, z_i^+)/\tau)}{\sum_{j=1}^{N} \exp(s(z_i, z_j^+)/\tau)}.$$

SimCSE uses this form with cosine similarity and in-batch negatives. Its unsupervised version produces two representations of the same sentence using independent dropout masks; its supervised version uses entailment pairs as positives and contradiction examples as hard negatives. [11]

DPR uses an analogous softmax ranking loss over a positive passage and negatives. E5 similarly trains text pairs with contrastive learning and in-batch negatives. [10,14]

Temperature controls how sharply score differences affect the softmax. A smaller τ magnifies differences in similarity, but the useful value is model- and training-dependent.

Important. Contrastive learning does not teach an abstract universal notion of "semantic similarity." It teaches geometry according to the positive/negative pairs, tasks, sampling strategy, and similarity function in the training data.

Research anchors: [10], [11], [13], [14], [18]

21 - WORKED CONTRASTIVE LOSS

One positive competes against two negatives

Suppose an anchor has cosine similarity

$$s^+ = 0.8$$

with its positive and similarities

$$s_1^- = 0.3, \qquad s_2^- = 0.0$$

with two negatives. Let $\tau = 0.5$.

The exponentiated scores are

$$e^{0.8/0.5} \approx 4.9530, \quad e^{0.3/0.5} \approx 1.8221, \quad e^{0/0.5} = 1.$$

The softmax probability assigned to the positive is

$$p^+ = \frac{4.9530}{4.9530 + 1.8221 + 1} \approx 0.6370.$$

The loss is

$$\ell = -\log p^+ \approx 0.4509.$$

If the positive similarity rises while negatives stay fixed, p^+ increases and the loss falls. If a hard negative becomes more similar to the anchor, it receives more probability mass and the loss rises.

This toy calculation shows why negative choice matters. The gradient is shaped not just by the positive pair but by the alternatives present in the denominator.

Important. Absolute similarity scores are not directly comparable across arbitrary embedding models. Temperature, normalization, model training, and score distributions all matter.

Connection to in-batch negatives

In the in-batch setting, this three-item denominator is the same idea as one row of the B x B query-document score matrix on the next page: one designated positive competes against the other batch items used as negatives.

Research anchors: [10], [11], [18]

22 - NEGATIVES AND HARD NEGATIVES

The denominator teaches the boundary

In retrieval training, most possible documents are irrelevant to a given query. The challenge is choosing negatives that teach useful distinctions without introducing too many false negatives.

DPR compared random negatives, BM25-retrieved negatives, and passages paired with other questions. It also showed how in-batch negatives reuse the other passages in a batch, producing a $B \times B$ score matrix from B query-document pairs. [10]

For

$$Q \in \mathbb{R}^{B \times d}, \qquad D \in \mathbb{R}^{B \times d},$$

the matrix

$$S = QD^{\top}$$

contains every query-document dot product in the batch. Diagonal pairs can be positives, while off-diagonal pairs serve as negatives if they are truly irrelevant.

Hard negatives are especially valuable because they look plausible under a weaker retriever or share surface features with the query. But an actually relevant "negative" creates contradictory supervision.

Modern objectives increasingly include false-negative filtering or richer negative pools. Qwen3 Embedding, for example, masks certain candidates when its similarity-based criterion suggests they may behave like false negatives. [18]

Mental model. Contrastive training quality depends as much on the structure of negatives as on the positive examples.

Research anchors: [10], [11], [14], [18]

23 - DENSE RETRIEVAL

Precompute documents, encode the query, search the vector space

A dense retriever builds an embedding for each document or passage, stores those vectors in an index, then embeds a query and searches for high-scoring vectors.

DPR formalized the classic dual-encoder pattern:

$$E_Q(q) \in \mathbb{R}^d, \qquad E_P(p) \in \mathbb{R}^d,$$

$$\text{sim}(q, p) = E_Q(q)^\top E_P(p).$$

The expensive passage encoder runs offline for the corpus. Online query cost is one query encoding plus vector search. [10]

This is why embedding quality and search-system quality are separate problems. A perfect exact nearest-neighbor search over bad embeddings still returns neighbors according to the wrong geometry. Conversely, a strong embedding model can be undermined by the wrong similarity metric, stale vectors, bad chunking, or overly aggressive approximate search.

Dense retrieval also does not make lexical retrieval obsolete. E5's analysis explicitly noted cases where BM25 retained advantages, especially for exact lexical matching and certain long-tail or long-document tasks. [14]

Important. An embedding model defines a scoring space. A vector index is an algorithm for searching that space. Do not confuse representation learning with index structure.

Research anchors: [10], [14]

24 - QUERY/DOCUMENT ASYMMETRY AND INSTRUCTIONS

The two sides do not always mean the same thing

A retrieval query and a document play different semantic roles. Many embedding models make that asymmetry explicit.

E5 uses one shared encoder but prepends different textual identifiers: “query:” for queries and “passage:” for passages. The paper reports that breaking the symmetry this way is important for some retrieval tasks. [14]

Newer instruction-aware embedding models generalize the idea. Qwen3 Embedding concatenates a task instruction with the query while leaving the document unchanged, allowing relevance to be conditioned on the requested task. [18]

This means a production pipeline must preserve the model’s input protocol. Encoding a corpus with the query template, omitting a required prefix, or changing an instruction after indexing can create a mismatch even though every vector has the expected dimension.

For symmetric tasks such as generic sentence similarity, the same textual role may be used on both sides. For asymmetric retrieval, query/document conventions can be part of the learned geometry.

Mental model. The string you embed is not always just the user’s raw text. Task instructions, prefixes, prompts, and document formatting can be part of the model contract.

Research anchors: [14], [18]

25 - EMBEDDING DIMENSION

Capacity, storage, bandwidth, and search cost

An embedding of dimension d stores d numbers. Holding datatype fixed, vector payload grows linearly with d.

For one million vectors stored as float32:

$$1{,}000{,}000 \times 1024 \times 4 \text{ bytes} \approx 3.81 \text{ GiB},$$

while 256 dimensions require approximately

$$0.95 \text{ GiB}.$$

That is a 4× reduction in raw vector payload. Real indexes add metadata, graph structures, quantization state, alignment, and other overhead.

Higher dimension can give a model more representational capacity, but "more dimensions is always better" is false. Retrieval quality, training, data, architecture, and index behavior all interact with dimension.

The dangerous shortcut is to truncate an arbitrary embedding vector after training and assume its first coordinates are a good lower-dimensional representation. Standard training does not guarantee that information is ordered that way.

Some modern models are explicitly trained for flexible dimension. Qwen3 Embedding reports Matryoshka support, and its official model cards expose user-selectable output dimensions within model-specific ranges. [18]

Important. Dimension reduction is safe only when the model or a validated post-processing method supports it. Matching array shape is not evidence that semantics were preserved.

Research anchors: [15], [18]

26 - MATRYOSHKA REPRESENTATIONS

Train useful prefixes instead of truncating blindly

Matryoshka Representation Learning (MRL) trains a high-dimensional vector so that selected prefixes are themselves useful representations. If

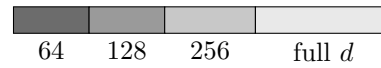
$$z \in \mathbb{R}^d,$$

then for chosen sizes $m \in \mathcal{M}$, the prefix

$$z_{1:m} \in \mathbb{R}^m$$

is explicitly optimized for the task. [15]

The original MRL paper uses nested dimensions, often spaced roughly logarithmically, so one model can support several accuracy/efficiency operating points without recomputing a completely different representation.



This is fundamentally different from PCA or taking a random subset of coordinates after training. MRL changes the training objective so useful information is packed into nested prefixes.

If each prefix is used with cosine similarity, normalize that prefix independently before scoring rather than reusing the norm of the full vector. [15]

Mental model. Flexible dimensions are a learned property, not a generic property of dense vectors.

Research anchors: [15], [18]

27 - ANISOTROPY AND REPRESENTATION GEOMETRY

A vector space can collapse into a narrow region

A useful embedding space is not guaranteed to fill $\mathbb{R}^d$ evenly. Contextual and sentence representations can become anisotropic: many vectors occupy a narrow cone or are dominated by a small set of directions. [11,21]

That matters because similarity then has a compressed dynamic range. If almost every pair has a high cosine similarity, separating truly related examples from unrelated ones becomes harder.

SimCSE connects contrastive learning with improved uniformity of the representation space. Its analysis argues that the contrastive objective can flatten the singular-value spectrum and make sentence representations more isotropic while preserving alignment of positive pairs. [11]

Anisotropy is not a universal scalar defect with one universal fix. Post-processing methods, normalization, model architecture, pretraining objectives, and embedding fine-tuning can all change the geometry.

Two useful concepts are:

- **alignment**: positives should be close;
- **uniformity**: unrelated samples should not all collapse into the same region.

Important. A high-dimensional vector can still have low effective geometric diversity. Dimension count alone does not tell you how the space is used.

Research anchors: [11], [21]

28 - SINGLE-VECTOR, MULTI-VECTOR, AND SPARSE REPRESENTATIONS

"Embedding" does not always mean one dense vector

The dominant mental model is one dense vector per text, but retrieval representations are broader.

Single-vector dense. A sentence or document is compressed into one vector. This is the classic SBERT/DPR/E5-style interface and is easy to index. [9,10,14]

Multi-vector / late interaction. A document can retain multiple token- or region-level vectors and defer some interaction until scoring. This preserves finer-grained information at greater storage and scoring cost.

Sparse learned representations. A model can produce a sparse weighted vocabulary-like vector rather than a dense low-dimensional vector.

BGE-M3 is a useful modern example because one model supports dense retrieval, sparse retrieval, and multi-vector retrieval. [17]

These representations solve different compression/interaction trade-offs. A single dense vector is maximally convenient for precomputation, but it must compress all relevant information into one point. Multi-vector approaches keep more local detail. Sparse representations preserve lexical dimensions in a learned form.

Important. Do not use "embedding model" as a synonym for "single dense vector model" when discussing modern retrieval systems.

Illustrative representation size

A 1,024-dimensional single-vector representation stores 1,024 scalar values per document. Retaining 128 token vectors of dimension 128 stores 16,384 values before compression - 16x as many. Real multi-vector systems may quantize, prune, or otherwise compress these representations, so this is a representation-size example, not a universal storage ratio.

Research anchors: [9], [10], [14], [17]

29 - MULTILINGUAL AND MULTIMODAL EMBEDDINGS

Shared spaces can cross languages and modalities

The same representation-learning idea can align objects that are not written in the same language, or not text at all.

A multilingual embedding model can train semantically corresponding texts from different languages to occupy compatible regions of one space. Modern models such as BGE-M3 and Qwen3 Embedding report multilingual training and evaluation across many languages. [17,18]

A multimodal model can align different data types. CLIP jointly trains an image encoder and a text encoder on image-text pairs so that matched images and descriptions receive compatible representations. This makes text-to-image and image-to-text retrieval possible without forcing both modalities through the same raw input encoder. [12]

The core pattern is still

$$f_a(x_a),\ f_b(x_b) \rightarrow \text{shared comparison space},$$

but each modality can have its own encoder and preprocessing.

The existence of a shared space does not mean the vectors are interchangeable for every task. The training pairs and objective determine what cross-modal or cross-lingual relationships are emphasized.

Mental model. Embedding is a representation interface. If training aligns two domains into a compatible geometry, similarity can bridge languages, modalities, or data types.

Research anchors: [12], [17], [18]

30 - EVALUATING EMBEDDINGS

One benchmark score cannot define a universal embedding

Embedding models are used for different tasks: semantic textual similarity, retrieval, classification, clustering, pair classification, reranking support, bitext mining, and more.

MTEB was created partly because evaluating only one task gives an incomplete picture. Its original benchmark covered eight task categories and found that no single method dominated every task in that evaluation. [16]

Different tasks require different metrics:

- STS often uses correlation between predicted similarity and human scores;
- retrieval uses ranking metrics such as Recall, MRR, nDCG, or task-specific variants;
- classification evaluates a classifier or probe built on embeddings;
- clustering measures whether the geometry supports meaningful grouping.

A retrieval model can be excellent even if its raw cosine scores are poorly calibrated as probabilities. A model that clusters topics well might not be the best asymmetric query-document retriever.

Benchmarks also depend on language, domain, text length, instructions, data contamination, and evaluation implementation. The current MTEB project has expanded far beyond the original text benchmark, including broader multilingual and multimodal evaluation, so leaderboard claims should always be date- and benchmark-scoped. [8]

Important. Choose an embedding model using the task and data distribution you actually care about, not one global average score.

Research anchors: [8], [16]

31 - EMBEDDINGS IN RAG

Embeddings choose candidates; they do not generate the answer

In a typical dense RAG pipeline:

1. split or otherwise define retrievable units;
2. encode those units into document embeddings;
3. build a vector index;
4. encode the user's query using the correct query protocol;
5. retrieve high-scoring candidates;
6. optionally rerank them;
7. send selected text to the generator.

The embedding model therefore affects which evidence reaches the LLM, but it is not the generator and it does not contain the full external corpus inside each query vector.

A document embedding is a lossy representation optimized for a similarity objective. It is useful for candidate selection because comparing vectors is cheaper than running a powerful cross-encoder or generative model jointly over every document.

This creates several separate failure modes: bad chunk boundaries, an embedding model that does not understand the domain, a query/document formatting mismatch, an approximate-search miss, or a reranker/generator failure.

Important. RAG is not "store knowledge in embeddings." The source text remains the evidence. Embeddings are a retrieval representation used to find candidate text.

Research anchors: [10], [14], [18]

32 - PRODUCTION COUPLING AND VERSIONING

Vectors are only comparable inside a compatible embedding space

A stored vector silently depends on many choices:

- model weights and exact version;
- tokenizer and text preprocessing;
- query/document prompts or instructions;
- pooling/readout rule;
- embedding dimension or truncation setting;
- normalization;
- output dtype or quantization;
- similarity metric.

Changing any of these can make new vectors incompatible with an existing index. Two models can both output 1024 dimensions yet inhabit unrelated coordinate systems.

This is why embedding-model upgrades often require re-embedding the corpus. A dimension match is not enough. Even a new checkpoint from the same family can rotate, rescale, or otherwise reshape the space.

Production systems should therefore version the embedding contract alongside the index. Useful metadata includes model identifier, revision, preprocessing template, role/instruction, output dimension, normalization convention, and metric.

Current embedding libraries expose many of these as configuration rather than universal defaults. Qwen3, for example, supports user-defined task instructions and flexible dimensions; Sentence Transformers exposes pooling and similarity modules. [18,20]

Mental model. An embedding index is coupled to the encoder that produced it. Treat encoder changes like schema changes, not cosmetic model swaps.

Research anchors: [18], [20]

33 - A TRANSPARENT PYTORCH IMPLEMENTATION

Lookup, masked mean pooling, and normalization

The following code shows three distinct stages: token lookup, pooling contextual states, and optional L2 normalization.

```
import torch
import torch.nn.functional as F

# 1) Token embedding lookup
V, d = 1000, 64
embedding = torch.nn.Embedding(V, d)
ids = torch.tensor([[4, 1, 7]]) # [B=1, T=3]
x = embedding(ids) # [1, 3, 64]

# 2) Imagine 'hidden' is the contextual Transformer output
hidden = x # pedagogical placeholder
mask = torch.tensor([[1, 1, 1]], dtype=torch.bool)

weights = mask.unsqueeze(-1).to(hidden.dtype)
pooled = (hidden * weights).sum(dim=1) / weights.sum(dim=1)
                                          # [1, 64]

# 3) Unit-normalize if the model contract expects it
z = F.normalize(pooled, p=2, dim=-1)
```

In a real text-embedding model, `hidden` would be produced by the contextual encoder, and the pooling/readout must match the model's training configuration. Some models use mean pooling, some a special token, some the last/EOS state, and some an additional projection.

Important. This code is pedagogical. Do not replace a model's documented pooling or prompt protocol with this generic mean-pooling recipe merely because the tensor shapes work.

Research anchors: [9], [18], [19], [20]

34 - COMMON IMPLEMENTATION MISTAKES

Most failures are contract mismatches, not vector arithmetic bugs

1. Pooling padding tokens. Mean pooling should generally respect the attention mask.

2. Mixing normalized and unnormalized vectors. This silently changes score semantics.

3. Using the wrong similarity metric. Cosine, dot product, and L2 can rank differently when norms vary.

4. Omitting query/document prefixes or instructions. Some models were explicitly trained with asymmetric input formatting.

5. Changing dimensions without MRL support. Arbitrary truncation need not preserve useful geometry.

6. Comparing vectors from different model versions. Same shape does not imply same coordinate system.

7. Reusing a corpus index after changing preprocessing. Tokenization, templates, and truncation can all change vectors.

8. Treating raw LM hidden states as a drop-in retrieval model. Good language-model representations are not automatically optimized for nearest-neighbor semantics.

9. Forgetting that long inputs may be truncated. The resulting vector may represent only the retained prefix or model-specific window behavior.

Mental model. Write the embedding contract down. Model, input format, readout, dimension, normalization, and metric should be explicit configuration, not tribal knowledge.

Research anchors: [9], [14], [15], [18], [20]

35 - COMMON MISCONCEPTIONS

Shortcuts that create the wrong mental model

Misconception	Correction
"A token ID is an embedding."	An ID selects a row. The vector row is the embedding.
"Each embedding dimension has one meaning."	Representations are generally distributed across dimensions and directions.
"The same word always has the same embedding."	Its input row may be fixed, but contextual hidden states depend on context.
"BERT/LLM hidden states are automatically good retrieval embeddings."	Similarity geometry usually needs a suitable readout and training objective.
"Cosine and dot product are the same."	They agree on unit-normalized vectors, not arbitrary vectors.
"More dimensions always improve retrieval."	Dimension trades capacity against storage/compute and depends on training.
"You can compare embeddings from different models if dimensions match."	Different encoders define different coordinate systems.
"RAG stores facts in vectors."	Vectors help retrieve source text; they are not the source text itself.

Important. The word "embedding" describes a representation, not one universal architecture, objective, pooling rule, or similarity metric.

Research anchors: [4], [9], [10], [15], [20]

36 - THE COMPLETE MENTAL MODEL

Embeddings in one chain

Embeddings are easiest to remember as a sequence of nine ideas.

1. **Discretize.** Tokenization maps text to token IDs.
2. **Look up.** IDs select learned rows from a vocabulary embedding matrix.
3. **Contextualize.** Transformer layers turn base rows into context-dependent hidden states.
4. **Read out.** A pooling or model-specific rule converts variable-length states into a fixed-size text vector when needed.
5. **Shape geometry.** Contrastive or task-specific training makes useful pairs score better than negatives.
6. **Choose a metric.** Dot product, cosine, and L2 only become meaningful with the model's normalization convention.
7. **Choose capacity.** Embedding dimension controls vector payload and can be flexible only when training supports it.
8. **Index.** Retrieval systems precompute document vectors and search them efficiently; the index is separate from the encoder.
9. **Version the contract.** Model, preprocessing, prompts, pooling, dimension, normalization, and metric must remain compatible.

Static word vectors, contextual token representations, sentence embeddings, dense retrievers, Matryoshka vectors, multilingual spaces, and multimodal spaces are all variations on the same broad idea: learn a representation in which downstream structure becomes easier to compute.

Mental model. An embedding is useful geometry learned for a purpose. To understand one, ask what was mapped, how the vector was produced, what objective shaped the space, and how vectors are compared.

Research anchors: [1]–[20]

REFERENCES

Primary papers and current implementation sources

1. Y. Bengio, R. Ducharme, P. Vincent, C. Jauvin. *A Neural Probabilistic Language Model*. JMLR 3, 2003.
2. T. Mikolov, K. Chen, G. Corrado, J. Dean. *Efficient Estimation of Word Representations in Vector Space*. arXiv:1301.3781, 2013.
3. J. Pennington, R. Socher, C. Manning. *GloVe: Global Vectors for Word Representation*. EMNLP 2014.
4. A. Vaswani et al. *Attention Is All You Need*. NeurIPS 2017. arXiv:1706.03762.
5. O. Press, L. Wolf. *Using the Output Embedding to Improve Language Models*. EACL 2017. arXiv:1608.05859.
6. M. Peters et al. *Deep Contextualized Word Representations*. NAACL 2018. arXiv:1802.05365.
7. J. Devlin, M.-W. Chang, K. Lee, K. Toutanova. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL 2019. arXiv:1810.04805.
8. MTEB project documentation and repository. Current benchmark scope, tasks, and evaluation tooling. Consulted September 2026.
9. N. Reimers, I. Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP 2019. arXiv:1908.10084.
10. V. Karpukhin et al. *Dense Passage Retrieval for Open-Domain Question Answering*. EMNLP 2020. arXiv:2004.04906.
11. T. Gao, X. Yao, D. Chen. *SimCSE: Simple Contrastive Learning of Sentence Embeddings*. EMNLP 2021. arXiv:2104.08821.
12. A. Radford et al. *Learning Transferable Visual Models From Natural Language Supervision*. ICML 2021. arXiv:2103.00020.
13. A. Neelakantan et al. *Text and Code Embeddings by Contrastive Pre-Training*. arXiv:2201.10005, 2022.
14. L. Wang et al. *Text Embeddings by Weakly-Supervised Contrastive Pre-training*. arXiv:2212.03533, 2022.
15. A. Kusupati et al. *Matryoshka Representation Learning*. NeurIPS 2022. arXiv:2205.13147.
16. N. Muennighoff et al. *MTEB: Massive Text Embedding Benchmark*. EACL 2023. arXiv:2210.07316.
17. J. Chen et al. *BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. arXiv:2402.03216, 2024.
18. Y. Zhang et al. *Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models*. arXiv:2506.05176, 2025.
19. PyTorch documentation. `torch.nn.Embedding`. Current documentation consulted September 2026.
20. Sentence Transformers documentation. Pooling, similarity functions, normalization, and embedding utilities. Current documentation consulted September 2026.
21. B. Li et al. *On the Sentence Embeddings from Pre-trained Language Models*. EMNLP 2020. arXiv:2011.05864.